Faculdade de Design,
Tecnologia e Comunicação
Universidade Europeia

StudySpot

Reserva de Salas de Estudo

Projeto Multidisciplinar L-EI | 5º Semestre

Carlos Alexandre Avelino Lima – 20240740 – L-EIF - 26/27 - D02 – 20240740@iade.pt

Daniel Alexandre Mbeya Paulo – 20240992 – L-EIF - 26/27 - D02 – 20240992@iade.pt

Filomeno João Chiquete Sabino – 20241963 – L-EIF - 26/27 - D02 – 20241963@iade.pt

Savio Netxitxi da Silva Casimira – 20240896 – L-EIF - 26/27 - D02 – 20240896@iade.pt

Grupo 6

Lisboa, 9 de Outubro de 2026

Índice

Link do Repositório no GitHub 3
Palavras-Chave 3
1. Resumo 3
2. Glossário e convenções 4
3. Introdução e motivação 4
4. Problema, objetivos e âmbito 4
5. Enquadramento nas unidades curriculares 5
6. Conceitos de base 5
7. Solução proposta 5
8. Requisitos 6
9. Arquitetura distribuída 6
10. Modelo de dados e interfaces 7
11. Segurança informática 7
12. Componente de Inteligência Artificial 7
13. Modelo de faltas 8
14. Engenharia de software 8
15. Stack tecnológico 8
16. Plano de trabalho, riscos e testes 8
17. Conclusão 9
Bibliografia 9

Link do Repositório no GitHub

https://github.com/carloslima11/StudySpot.git

Palavras-Chave

Reserva de salas, Sistemas distribuídos, Tolerância a faltas, Segurança informática, Autenticação de dois fatores, Inteligência artificial, Problema de satisfação de restrições, Internet das coisas, Replicação de dados, Consistência transacional, Exclusão mútua.

1. Resumo

O StudySpot é um sistema distribuído de reserva de salas de estudo concebido no âmbito do Projeto de Desenvolvimento de Software. O projeto surge para dar resposta aos problemas de sobreposição de reservas, falta de segurança na autenticação e falhas em servidores centralizados que comprometem a disponibilidade do serviço.

A solução idealizada materializa-se numa aplicação web onde os utilizadores se autenticam com uma palavra-passe e um código de dois fatores (TOTP). As reservas submetidas são avaliadas de forma automática por um algoritmo de Inteligência Artificial que verifica restrições de horários e capacidade, enquanto dispositivos IoT instalados nas salas validam fisicamente a ocupação do espaço. A arquitetura técnica apoia-se em serviços executados em *containers* Docker, suportados por um *load balancer* (Nginx), uma base de dados relacional replicada (PostgreSQL) e coordenação em memória (Redis).

2. Glossário e convenções

Este capítulo clarifica a terminologia técnica adotada ao longo do relatório para garantir uma leitura inequívoca e padronizada:

- **Falta, erro, falha:** Falta é a causa de um problema; erro é o estado incorreto; falha é a interrupção do serviço;
- **Serviços sem estado (stateless):** Serviços que não guardam informação local, delegando a persistência para a base de dados;
- **TOTP:** *Time-based One-Time Password*, utilizado para a autenticação de dois fatores;
- **CSP:** Problema de Satisfação de Restrições (*Constraint Satisfaction Problem*), o modelo algorítmico utilizado na validação;
- **Idempotência:** Garantia de que repetir um pedido (por exemplo, devido a falhas de rede) não duplica o seu efeito no sistema.

3. Introdução e motivação

O presente projeto surge da necessidade de otimizar a alocação de salas de estudo universitárias, um recurso altamente procurado em épocas de avaliação. A gestão manual ou em bases de dados simples não previne que dois estudantes reservem o mesmo espaço em simultâneo. Tecnicamente, a construção desta solução motiva-se pela oportunidade de resolver desafios clássicos de sistemas distribuídos e de segurança informática, integrando de forma coesa as várias unidades curriculares do semestre.

4. Problema, objetivos e âmbito

A gestão de espaços físicos numa universidade enfrenta diversos desafios operacionais e tecnológicos que necessitam de resolução arquitetural.

Os problemas centrais incluem a concorrência (pedidos simultâneos geram conflitos), a autenticação fraca (dependência de passwords), a indisponibilidade (falha de um servidor paralisa o sistema) e a ocupação ineficiente (salas reservadas sem ocupação real). O objetivo geral é criar uma plataforma tolerante a falhas que combata estes problemas, garantindo o login seguro, a validação de pedidos sem sobreposição e a integração de sensores IoT para libertação automática de reservas "fantasma".

5. Enquadramento nas unidades curriculares

O desenvolvimento desta plataforma integra conhecimentos transversais adquiridos durante o 5.º semestre.

- **Sistemas Distribuídos:** Foco na replicação, redundância, concorrência e modelo de faltas;
- **Segurança Informática:** Aplicação de 2FA, derivação segura de credenciais e comunicações cifradas;
- **Inteligência Artificial:** Modelação e resolução do algoritmo CSP para atribuição de reservas;
- **Engenharia de Software:** Coordenação do ciclo de vida, planeamento ágil e modelação de requisitos.

6. Conceitos de base

Para a conceção do sistema, foram aplicados vários princípios teóricos fundamentais da Engenharia Informática.

Destaca-se o modelo de falhas por paragem (*crash-recovery*), assumindo que os componentes podem falhar e recuperar posteriormente. Adota-se o princípio da replicação de base de dados para garantir consistência nas leituras, e utilizam-se *locks* distribuídos acoplados a restrições nativas da base de dados (*fencing*) para impedir tecnicamente reservas duplicadas.

7. Solução proposta

A plataforma idealizada permite a interação entre utilizadores e hardware físico de forma fluida.

Os estudantes interagem com a interface para efetuar login, visualizar salas e submeter reservas. O Nó processa o pedido e aplica o bloqueio transacional, se aprovado, a reserva é registada. Paralelamente, leitores IoT registam a presença física na sala (*check-in*). Uma rotina automática do sistema encarrega-se de limpar as reservas cujo *check-in* não ocorreu dentro da janela de tempo permitida, realocando o espaço.

8. Requisitos

O sistema obedece a um conjunto rigoroso de requisitos essenciais para o seu funcionamento seguro e fiável.

Ao nível funcional, o sistema deve garantir o registo com 2FA, submissão de reservas com identificação de idempotência e o processamento automático de eventos IoT. Ao nível não funcional, exige-se que a plataforma não possua pontos únicos de falha nas camadas aplicacionais e garanta a integridade matemática de que existe apenas uma reserva válida por cada sala e horário.

9. Arquitetura distribuída

A arquitetura do sistema foi desenhada para eliminar pontos únicos de falha nas camadas de serviços e assegurar elevada escalabilidade.

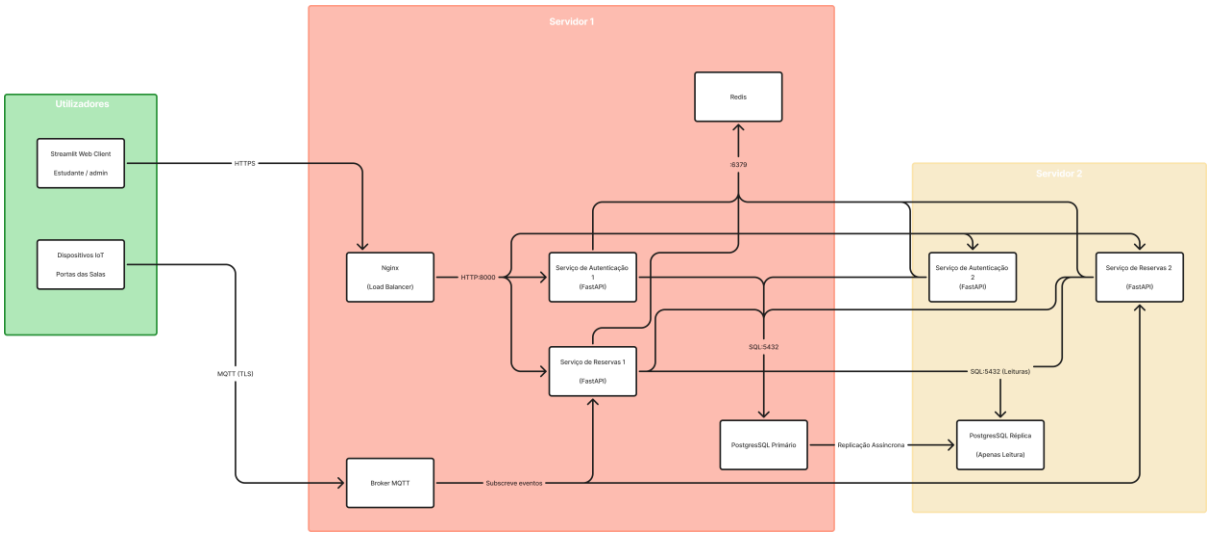


Figura 1 *- Esquema da arquitetura distribuída e fluxo de comunicações da plataforma StudySpot.*

A infraestrutura assenta num *Load Balancer* (Nginx) que distribui o tráfego web pelas instâncias ativas do Serviço de Autenticação e do Serviço de Reservas. Como os serviços são independentes de estado (*stateless*), a coordenação de *locks* e de sessões de utilizador é delegada ao Redis. A persistência de dados opera num PostgreSQL configurado com um nó Primário (escritas e leituras) e um nó Réplica (leituras). Por fim, um Broker MQTT independente gere as mensagens assíncronas enviadas pelos dispositivos IoT.

10. Modelo de dados e interfaces

A persistência da informação e a comunicação entre sistemas baseiam-se em modelos e rotas estruturadas.

A base de dados relacional organiza-se em tabelas críticas como utilizadores, segredos_totp e reservas. O isolamento da concorrência é efetuado através de uma restrição inquebrável do tipo UNIQUE na tabela que mapeia os horários ocupados. A comunicação web realiza-se via API REST, enquanto os eventos físicos comunicam exclusivamente via mensagens MQTT estruturadas em JSON.

11. Segurança informática

A proteção dos utilizadores e da infraestrutura é garantida através de múltiplas camadas de defesa mitigadoras de ameaças comuns.

O sistema não armazena palavras-passe em texto limpo, utilizando derivação pesada, e cifra os segredos utilizados para a geração do TOTP. A limitação de acessos repetidos evita ataques de força bruta, enquanto o princípio de minimização de dados no IoT impede que a plataforma armazene identificadores físicos diretos dos estudantes.

12. Componente de Inteligência Artificial

A otimização das reservas é suportada por algoritmos de validação avançados.

A lógica da plataforma modela os pedidos num Problema de Satisfação de Restrições (CSP). O algoritmo avalia condicionantes de capacidade, horário de funcionamento e limites de horas diárias permitidas por estudante. Esta validação inteligente permite não só aceitar reservas viáveis, como também reconstruir horários com precisão quando turmas em lista de espera transitam para salas recém-libertadas.

13. Modelo de faltas

A fiabilidade do sistema é assegurada por um modelo claro que antevê falhas e dita o comportamento de recuperação aplicacional.

O sistema suporta a paragem técnica de uma instância de backend ou da base de dados réplica sem indisponibilizar a plataforma aos utilizadores. Em cenários extremos onde falhe a base de dados principal ou o *cache* Redis, a aplicação adota o princípio de "Falha Segura", recusando operações e devolvendo uma mensagem de erro clara em vez de permitir marcações corrompidas.

14. Engenharia de software

A coordenação do projeto baseia-se em metodologias ágeis e ferramentas rigorosas de versionamento de código.

Utiliza-se o GitHub para garantir um histórico auditável, segmentando o desenvolvimento em *branches* (main para entregas formais, develop para integração contínua). A organização de tarefas é efetuada no ClickUp, e a qualidade de desenvolvimento é suportada por práticas de testes e revisão entre a equipa.

15. Stack tecnológico

A implementação da plataforma recorre a um conjunto de tecnologias modernas adequadas à operação em sistemas distribuídos.

A orquestração do ambiente decorre em Docker e Docker Compose. O *backend* assenta em Python com o ecossistema FastAPI e SQLAlchemy, suportado por bases de dados PostgreSQL e Redis. A interface com o utilizador é construída na *framework* Streamlit, e a componente IoT depende de protocolos geridos pelo *broker* Eclipse Mosquitto.

16. Plano de trabalho, riscos e testes

A execução do projeto está dividida em fases bem definidas, com identificação proativa de riscos para evitar surpresas no decorrer do semestre.

O cronograma define a análise teórica para a 1.ª entrega, seguindo-se o desenvolvimento do protótipo e dos *containers* Docker para a 2.ª entrega. A implementação final e os testes de indução de falhas serão concluídos na 3.ª entrega. Identificam-se como principais riscos a complexidade inicial de configuração do *failover* do PostgreSQL e a gestão de comunicação do *broker* IoT.

17. Conclusão

Esta primeira fase de conceptualização permitiu consolidar a base teórica e estrutural da plataforma StudySpot.

Definiu-se uma arquitetura onde o balanceamento de carga, os serviços *stateless* e a persistência replicada fornecem garantias ativas contra problemas de disponibilidade e concorrência habituais em contextos universitários. Com os cenários de faltas documentados e os limites do sistema claramente estabelecidos, a equipa encontra-se preparada para iniciar a prototipagem prática do código e dos *containers* na fase subsequente do projeto.

Bibliografia

1. G. Coulouris, J. Dollimore, T. Kindberg e G. Blair, *Distributed Systems: Concepts and Design*, 5.ª ed., Pearson, 2011.
2. S. Russell e P. Norvig, *Artificial Intelligence: A Modern Approach*, 4.ª ed., Pearson, 2020.
3. D. M'Raihi et al., *RFC 6238: TOTP: Time-Based One-Time Password Algorithm*, IETF, 2011.
4. D. M'Raihi et al., *RFC 4226: HOTP: An HMAC-Based One-Time Password Algorithm*, IETF, 2005.
5. OASIS, *MQTT Version 3.1.1*, 2014.
6. PostgreSQL Global Development Group, *PostgreSQL Documentation: High Availability, Load Balancing, and Replication*.
7. Redis, *Distributed Locks with Redis*.
8. M. Kleppmann, "How to do distributed locking", 2016.
9. M. Kleppmann, *Designing Data-Intensive Applications*, O'Reilly, 2017.
10. S. Gilbert e N. Lynch, "Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services", *ACM SIGACT News*, 2002.
11. A. Avizienis, J.-C. Laprie, B. Randell e C. Landwehr, "Basic concepts and taxonomy of dependable and secure computing", *IEEE Transactions on Dependable and Secure Computing*, 2004.
12. OWASP, *Password Storage Cheat Sheet*.
13. NIST, *SP 800-63B: Digital Identity Guidelines — Authentication and Lifecycle Management*.
14. Stripe, *Documentation: Idempotent requests*.
15. Regulamento (UE) 2016/679 (Regulamento Geral sobre a Proteção de Dados).
16. A. K. Mackworth, "Consistency in networks of relations", *Artificial Intelligence*, 1977.
17. A. Shostack, *Threat Modeling: Designing for Security*, Wiley, 2014.